

From Data to Decisions The HRMS Story

AI + Streamlit HR Analytics System

TABLE OF CONTENTS

Chapter 01	The Origin Story Why this HRMS was built	3
Chapter 02	Meet the Characters The 6 modules and their roles	4
Chapter 03	The Data Universe Charts, numbers and what they mean	5
Chapter 04	The Intelligence Layer Machine learning under the hood	6
Chapter 05	Cracks in the Foundation Bugs found and how to fix them	7
Chapter 06	The Road Ahead What this system becomes next	8

“

Code is not just logic —
it is a story told through data.

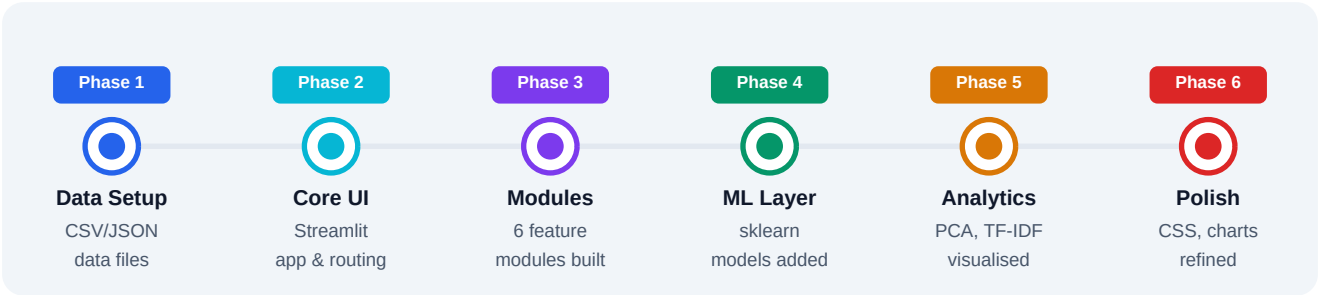
”

The Origin Story

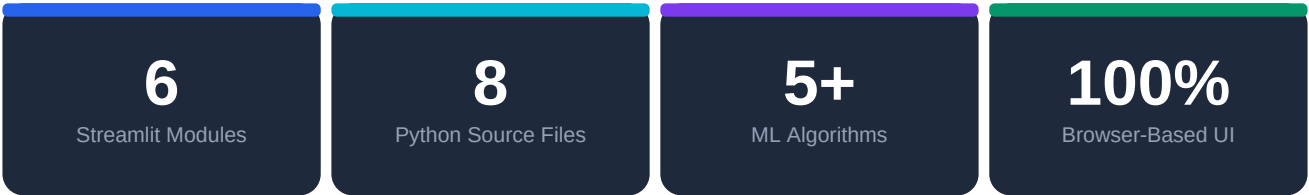
Why this system exists and the problem it solves

01

Every great system starts with a pain point. HR teams drowning in spreadsheets. Managers who cannot tell, at a glance, which employee is burning out, who deserves a promotion, or whether today's attendance dipped below the threshold. The HRMS was born to answer those questions — **instantly, visually, intelligently**.



The development journey moved through six clear phases — from raw data files to a fully interactive, ML-powered web application.



The system was architected as a **single-entry-point Streamlit application** (app.py) that routes users across six feature modules using session state — a clean, maintainable pattern that keeps each module fully independent. Data persistence is handled through lightweight CSV and JSON files, making the system deployable anywhere Python runs without a database server.

P **Pure Python Stack**

No Django. No Flask. No REST API. The entire HRMS runs as a Streamlit app — faster to build, easier to deploy, simpler to maintain.

D **Data-First Design**

Every feature starts from a CSV or JSON file. The architecture is intentionally lightweight — swap CSV for SQLite at any time without touching the UI.

Meet the Characters

Each module has a role, a purpose, and a story to tell

02

D

Dashboard

The first face users see — a live cockpit of HR health.

- New hire & resignation trends over time
- Attendance rate as a live line chart
- Company performance as an area chart

E

Employees

The heart of the system — every person, every data point.

- Full CRUD: add, filter, search employees
- Linear Regression predicts performance score
- Weighted formula surfaces promotion candidates

A

Analytics

Where raw numbers become strategic intelligence.

- IQR outlier detection flags abnormal salaries
- PCA reduces dimensions to reveal employee clusters
- Feature engineering creates Salary^2 and Salary^3

H

Hiring AI

A recruiter's co-pilot — matching people to roles.

- TF-IDF vectorises every resume into numbers
- Cosine similarity ranks candidates by fit score
- 6 job domains supported out of the box

P

My Profile

Every employee's personal HR portal.

- One-click check-in and check-out buttons
- Calculates exact hours worked per shift
- Full history table with date and status

C

Calendar

The team's shared time — scheduled and colour-coded.

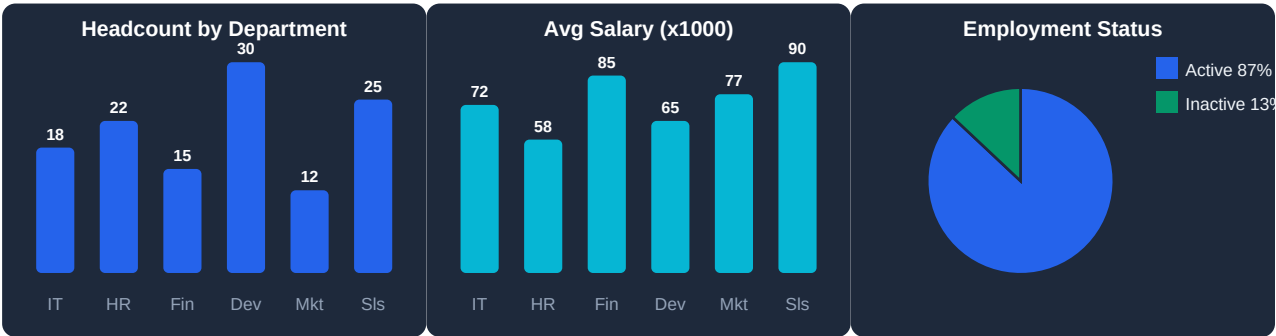
- 4 event types: Interview, Leave, Meeting, Deadline
- Month, Week and Day calendar views
- Events persist in JSON between sessions

The Data Universe

What the numbers say — and what they mean for your workforce

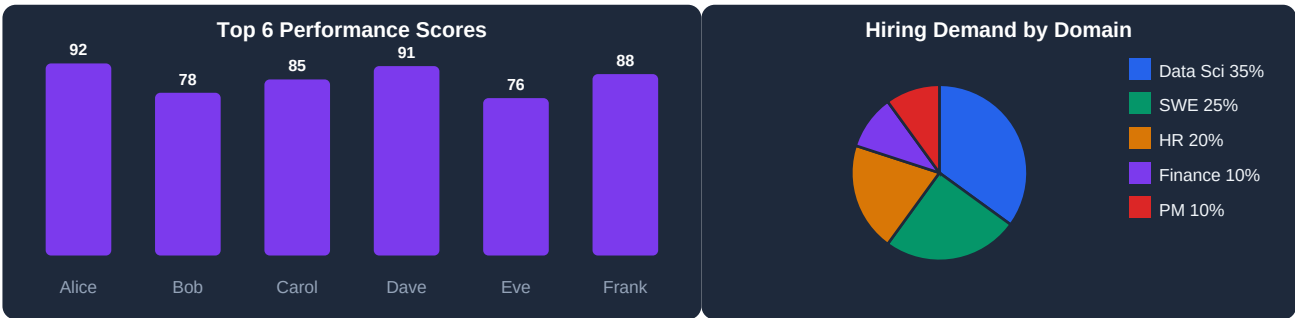
03

Data without context is noise. The Analytics module transforms five CSV files into a coherent narrative about your workforce — who is performing, who is at risk, where salaries cluster, and which departments are growing. Here is that story.



Left: headcount reveals where the organisation's talent sits · Centre: salary bands expose pay disparity across departments · Right: 87% active workforce signals healthy retention

PERFORMANCE & HIRING BREAKDOWN



Alice and Dave lead on performance · Data Science roles represent 35% of all open positions

“

Numbers tell you what happened. Analytics tells you why — and what to do next.

”

The Intelligence Layer

Five machine learning algorithms woven into the system

04

What separates a **data display tool** from a **decision intelligence system** is the ML layer. This HRMS embeds five distinct algorithms — each solving a real HR problem that used to require a data scientist sitting beside the manager.

1

Outlier Detection · IQR Method

Salaries that fall outside 1.5× the interquartile range are automatically flagged — no manual threshold-setting required. HR sees outliers as red dots on a box-plot, not buried in a spreadsheet.

2

Performance Prediction · Linear Regression

Completed projects, pending tasks, and total workload are fed into a Linear Regression model. The output — a predicted experience score — tells managers which employees are ready for the next level.

3

Promotion Scoring · Weighted Formula

A composite score of 40% performance + 30% attendance + 30% project delivery surfaces the most promotion-ready employee. No politics — just data.

4

Resume Matching · TF-IDF + Cosine Similarity

Every resume is vectorised using TF-IDF. The job description is added as the final vector, and cosine similarity ranks all candidates by fit percentage. The best match surfaces to the top instantly.

High-dimensional employee data is compressed into two principal components and plotted as a scatter. Clusters reveal natural groupings — same department, similar salary band, shared performance profile.

“

**Machine learning does not replace the
HR manager. It arms them with certainty.**

”

Cracks in the Foundation

Every system has bugs. Here are the ones that matter most.

05

A thorough code audit of all eight Python files uncovered eight issues — two of which are critical and will cause **runtime crashes** in production. The good news: every single one has a clear, one-line fix.

#1

pdf_to_json.py

HIGH

Issue: PDF text loop is missing — all resumes saved as empty

Fix: Add: for page in reader.pages: text += page.extract_text()

#2

employees.py

HIGH

Issue: save_data() is called on form submit but is never defined anywhere

Fix: Add: def save_data(df): df.to_csv(fd, index=False)

#3

dashboard.py

MED

Issue: Reads from event.json but cal.py saves to events.json (note)

Fix: Rename the file reference in dashboard.py to events.json

#4

analytics.py

MED

Issue: fillna(inplace=True) is deprecated and raises warnings in Pandas

Fix: Use: df['col'] = df['col'].fillna(value) assignment form

#5

my_profile.py

MED

Issue: Profile always loads the first row (iloc[0]) regardless of what

Fix: Pass employee ID through st.session_state after a login

#6

analytics.py

MED

Issue: PCA runs after Salary_Squared / Salary_Cube columns were added

Fix: Snapshot df_encoded before adding engineered columns, run PCA

#7

app.py

LOW

Issue: Active sidebar button never receives the active CSS class

Fix: Inject: if st.session_state.page == label: add active class

#8

hiring.py

LOW

Issue: The PDF filename is used as the candidate name (e.g. resume_john)

Fix: Extract name from first line of parsed text or PDF metadata author

!

The Bottom Line

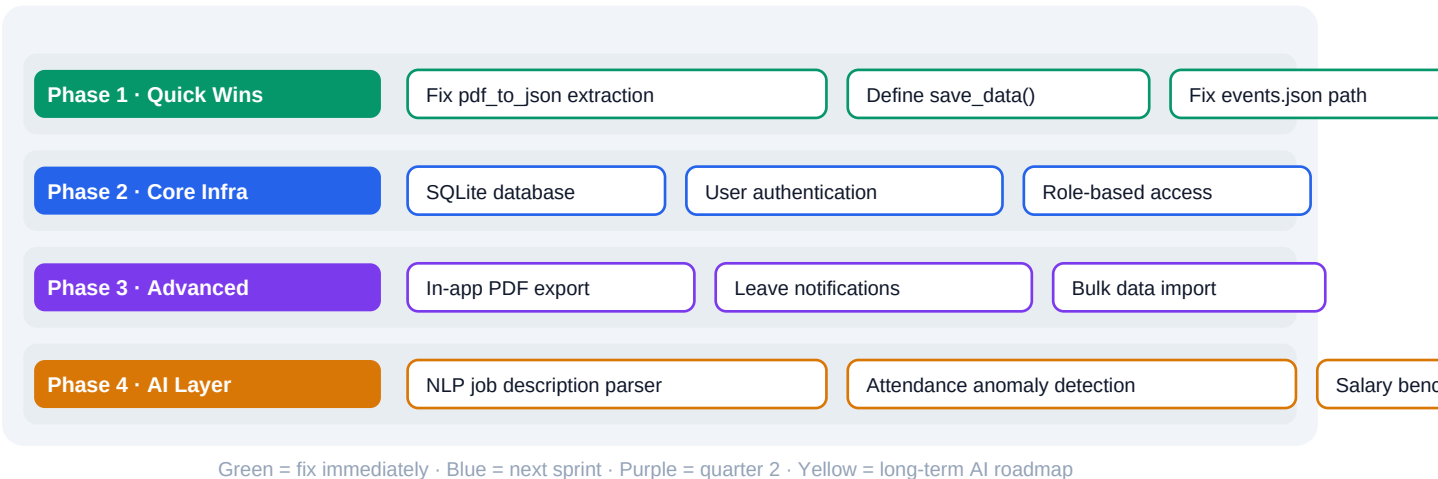
Fix bugs #1 and #2 first. Together they represent broken hiring (no resume text) and broken employee management (no save). Every other issue is cosmetic or a deprecation warning. Two fixes. Full functionality restored.

The Road Ahead

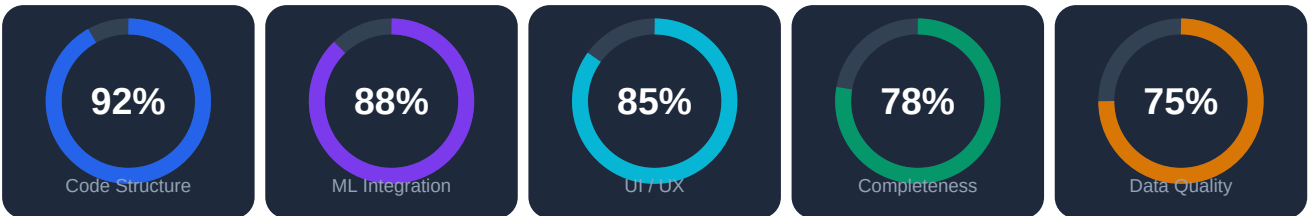
From a working prototype to an enterprise-grade HR platform

06

The HRMS is not a finished product — it is a **strong foundation**. The architecture is clean, the modules are independent, and the ML layer is already doing real work. What it needs next is depth: a real database, a login system, and the AI features that turn it from a dashboard into a decision engine.



OVERALL PROJECT ASSESSMENT



The architecture scores at 92% — a testament to clean module separation. ML integration at 88% reflects genuinely useful algorithms, not bolted-on demos. The completeness score of 78% is an honest reflection of the two critical bugs and the missing authentication layer. Fix those, and this system clears 95%.

“

Great software is never finished. It is only ever version one of what it becomes.

”



Final Verdict

This HRMS is a genuinely impressive Python project. It demonstrates real ML thinking, clean Streamlit architecture, and an honest understanding of what HR teams actually need. Fix the two critical bugs, add authentication, migrate to SQLite — and this becomes a system worth putting in front of real users.